

Topic 40 — Hugging Face: Platform, Access & CLI Setup

GitHub for AI models — signup, gated access, tokens, and the new **hf** CLI.



New to Hugging Face



Intermediate



Expert



Interview Q&A

Read this first — your course is out of date

Most tutorials say `huggingface-cli login`. **That command is dead.** The CLI was renamed to **hf** in July 2025, and `huggingface-cli download` was **completely removed** in `huggingface_hub` v1.0. This PDF uses the current commands.

Layer 1 — What is Hugging Face

The one-line answer



One line

Hugging Face is GitHub, but for AI models.

GitHub	Hugging Face
Stores code repositories	Stores model repositories
<code>git clone</code> a repo	<code>hf download</code> a model
Public and private repos	Public, private, and gated models
Issues, PRs, discussions	Discussions, community tabs
GitHub Actions	Spaces (hosted demos)
npm / PyPI for packages	The Hub for models, datasets, demos

When Meta releases Llama, or Google releases Gemma, they publish it **on Hugging Face**. It's where the weights live.

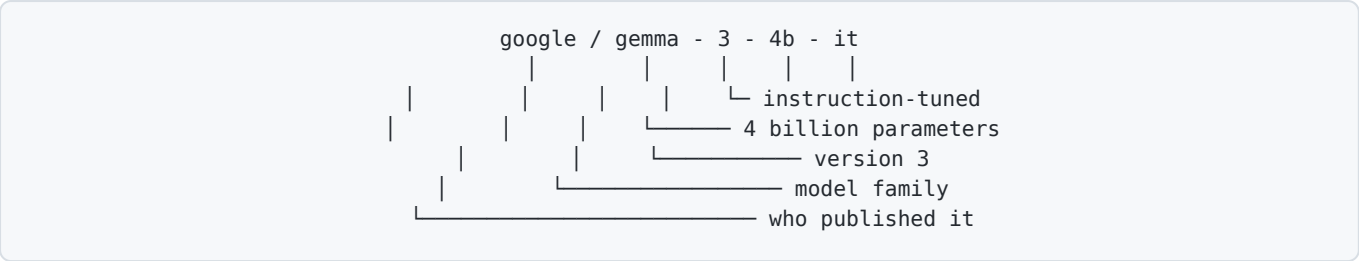
Three things live there

Thing	What it is	Example
Models	The actual trained weights	<code>google/gemma-3-4b-it</code>
Datasets	Training / evaluation data	<code>squad</code> , <code>imdb</code>

Thing	What it is	Example
Spaces	Live web demos anyone can try	A running chatbot in your browser

Notice the naming: `google/gemma-3-4b-it` is `owner/model-name` — exactly GitHub's `owner/repo` format.

Decoding a model name



 **That -it suffix matters a lot**

Base (gemma-3-4b) — raw text prediction. Give it *"The capital of France is"* and it continues. It does **not** understand "answer my question."

Instruction-tuned (gemma-3-4b-it) — fine-tuned on conversations. Understands system/user/assistant roles.

You almost always want -it. Base models are for people doing their own fine-tuning.

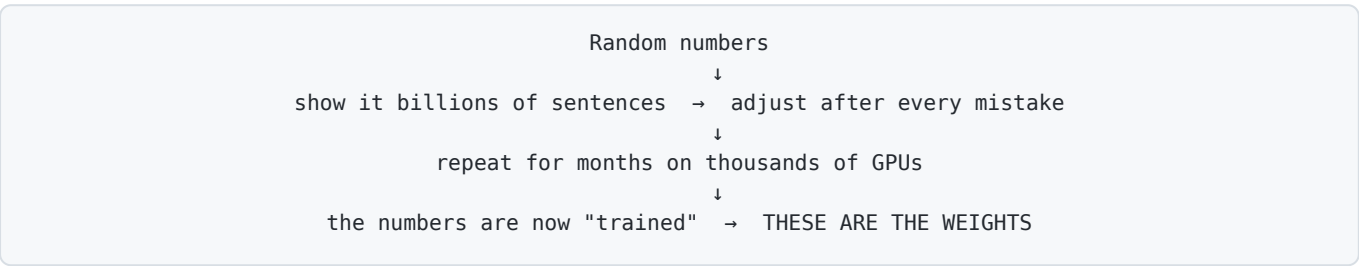
Sidebar — What are "weights"?

 **The word used everywhere, rarely defined**

Weights are the numbers that make a model work. They are the entire trained "brain," stored as a giant file.

When you run `ollama pull gemma:2b` and 1.7 GB downloads — **that download is the weights**. Nothing else. There is no program in there, just billions of numbers.

Where they come from



The adjusting is **training**. The final numbers are **the weights**. Google spent millions of dollars in compute producing the numbers in `gemma-3-4b-it` — that file *is* the product.

What "4 billion parameters" means

Parameters and weights are the same thing. `gemma-3-4b` means the file contains **4 billion numbers**. That's why the size table from Topic 34 works out — it's just multiplication:

Parameters	Bytes each (fp16)	File size
2 billion	2	~4 GB
4 billion	2	~8 GB

Parameters	Bytes each (fp16)	File size
7 billion	2	~14 GB

And it's why **quantization** shrinks things — store each number in 4 bits instead of 16 and the file drops to a quarter. You're not removing numbers, you're storing each one *less precisely*.

A programmer's framing

```
# What you're used to
Code (logic)
+
Data (the database)
```

```
# What a model is
Architecture (how numbers flow)
+
Weights (the numbers)
```

The architecture is **small** — a few hundred lines of Python in `transformers` . It's public, and Gemma's is broadly similar to Llama's. **The weights are the valuable part**. Two models can share identical architecture and behave completely differently because their weights differ.

A tiny concrete example

```
def predict(x):
    return w * x + b          # w and b are weights
```

Weights	predict(5) gives
<code>w = 2, b = 0</code>	10
<code>w = 7, b = 3</code>	38

✔ **Same code. Different weights. Different output.**

Training means: show it examples, and nudge w and b until the outputs are right. A language model is that idea — except with **4 billion ws** arranged in layers, and the training examples are most of the internet.

Why they're "just numbers"

```
[0.0234, -0.1891, 0.5522, 0.0011, -0.9834, ...]
```

Four billion of those. No words, no rules, no if-statements about grammar. All the "knowledge" is encoded in the specific values — which is also why nobody can fully explain *why* a model gives a particular answer.

Everything from Phase 5 now reads more clearly

What you saw	What it actually meant
"Models are 2–5 GB"	The weight files are that big
<code>/root/.ollama</code> volume	Where weight files are stored
"Loads into RAM on first request"	Reading 2 GB of numbers from disk into memory
Cold-start penalty	Re-reading those numbers after unload
<code>.safetensors</code>	A file format for storing weights safely
Quantization <code>q4_K_M</code>	Each number stored in 4 bits instead of 16
Content-addressed blobs	Two models sharing a weight file store it once

What you saw	What it actually meant
"The image ships the engine, not the fuel"	Ollama's Docker image has the code, not the weights

Two related terms

Term	Meaning
Inference	Using trained weights to generate output. What you've been doing. Cheap, runs on your laptop.
Training / fine-tuning	<i>Changing</i> the weights. Expensive, needs GPUs. Fine-tuning = taking Google's weights and nudging them on your own data.

 **Why the model can't remember your name**

Every model you've run has been **inference only**. The weights never change during a conversation — which is exactly why memory has to live in the messages array (Topic 39), not in the model.

Why you need it, given Ollama already works

Ollama	Hugging Face
~100 curated models	1.5 million+ models
Chat models only, mostly	Every task: vision, speech, embeddings, classification
One format (GGUF)	Every format
Run models	Run and fine-tune models
Zero configuration	Full control over everything

 **The one-sentence summary**

Ollama is the easy path. Hugging Face is the full toolbox.

Real use cases

Use case	What you'd use
Chat with a model in Python	<code>transformers</code> + a <code>-it</code> model
Turn text into vectors for RAG	<code>sentence-transformers</code>
Fine-tune on company data	<code>transformers</code> + <code>peft</code> (LoRA)
Read text from images	A vision model like <code>trocr</code>
Speech to text	<code>whisper</code>
Demo something publicly	Spaces
Get a model into Ollama	Download GGUF from HF, import it

That last row is worth noting — **many Ollama models originate on Hugging Face**. You can pull a GGUF file from HF and run it in Ollama directly.

Layer 2 — Getting access to Gemma 3 4B IT

Step 1 — Sign up

Go to huggingface.co → **Sign Up**. Free, email + password.

⚠️ Verify your email — this is required

Gated models will **not** work with an unverified account. Also fill in your profile with a real name; Google's Gemma approval is automatic, but a blank profile occasionally trips manual review.

Step 2 — What "gated" means

Most models you just download. Some — Gemma, Llama, Mistral's newer ones — are **gated**: you must accept a license before getting the weights.

💡 This is not a paywall — it's free

The publisher wants (a) you to have agreed to their acceptable-use terms, and (b) a record of who downloaded it, for export-control compliance.

Gemma 3 is gated. Skip this step and every download fails with a `401` or `403`.

Step 3 — Accept the Gemma license

1. Go to <https://huggingface.co/google/gemma-3-4b-it>
2. You'll see a banner: *"You need to agree to share your contact information to access this model"*
3. Read the Gemma Terms of Use
4. Click **Acknowledge license / Agree and access repository**

✅ How to check it worked

If you can see the **"Files and versions"** tab with `.safetensors` files listed, you have access. If you still see the license banner, you don't. Approval for Gemma is **usually instant** — just refresh.

⚠️ Acceptance is per model repo, not per family

Accepting `gemma-3-4b-it` does **not** give you `gemma-3-12b-it`. Each one needs its own click.

Step 4 — Install the CLI

The CLI ships inside the `huggingface_hub` Python package:

```
pip install -U huggingface_hub  
  
hf version
```

⚠️ hf: command not found?

Restart your terminal. The PATH needs refreshing after install.

If you're using `uv` from Topic 38:

```
uv add huggingface_hub
uv run hf version
```

Step 5 — The naming change your course got wrong

The CLI was renamed from `huggingface-cli` to `hf`, with the structure reorganised to follow the pattern `hf <resource> <action>`. Every tutorial written before mid-2025 uses the old commands:

❌ Old (removed)	✅ Current
<code>huggingface-cli login</code>	<code>hf auth login</code>
<code>huggingface-cli whoami</code>	<code>hf auth whoami</code>
<code>huggingface-cli logout</code>	<code>hf auth logout</code>
<code>huggingface-cli download</code>	<code>hf download</code>
<code>huggingface-cli upload</code>	<code>hf upload</code>
<code>huggingface-cli scan-cache</code>	<code>hf cache scan</code>

The full command set:

```
hf auth      # login, logout, whoami, switch, list
hf download  # get files from the Hub
hf upload    # push files to the Hub
hf repo      # create/manage repos
hf cache     # manage local cache
hf jobs      # run scripts on HF infrastructure (Pro)
hf version
hf env
```

Step 6 — Create an access token

A token is a **password for programs**. Never use your actual password in code.

1. Log in at huggingface.co
2. Click your **avatar** (top right) → **Settings**
3. Left sidebar → **Access Tokens**
4. **+ Create new token**

Token type	Use it when
Read	Downloading models — this is what you want
Write	Uploading your own models / datasets
Fine-grained	Production: scope to specific repos only

 **Copy it immediately**

It starts with `hf_...` and is shown **exactly once**. Close the dialog without copying and you must delete it and make a new one.

Treat it like a password. Never commit it to Git, never bake it into a Docker image, never share it in a screenshot. If it leaks, revoke it from the same settings page.

Step 7 — Log in

```
hf auth login
```

Enter your token (input will not be visible):

💡 **Your terminal shows nothing while you type**

That's deliberate, not a bug. Paste and press Enter.

Then it asks `Add token as git credential? (Y/n)` — say **n** unless you plan to push models via Git. Verify with:

```
hf auth whoami
```

Non-interactive version, for scripts and CI:

```
hf auth login --token hf_xxxxxxxxxxxx
```

Or via environment variable — cleanest for Docker:

```
export HF_TOKEN=hf_xxxxxxxxxxxx # Mac/Linux
setx HF_TOKEN hf_xxxxxxxxxxxx # Windows
```

The libraries read `HF_TOKEN` automatically. No login step needed.

Step 8 — Where the token gets stored

```
~/.cache/huggingface/token # Mac/Linux
C:\Users\<you>\.cache\huggingface\token # Windows
```

Plain text. Anyone with access to your machine can read it. Fine for a personal laptop; on a shared server, use `HF_TOKEN` from a secrets manager instead.

Step 9 — Download the model

```
hf download google/gemma-3-4b-it

# ...or into a specific folder
hf download google/gemma-3-4b-it --local-dir ./gemma-3-4b-it
```

Roughly **8 GB** for the 4B model. Grab a coffee.

```
hf download google/gemma-3-4b-it --exclude "*.gguf" "*.pth"
```

✅ **Why `--exclude` is genuinely useful**

Many repos ship the **same weights in three formats**. Downloading all of them triples your wait for nothing.

Step 10 — Use it in Python

```
pip install transformers torch accelerate
```

```
from transformers import pipeline

chat = pipeline(
    "text-generation",
    model="google/gemma-3-4b-it",
    device_map="auto",
)

messages = [
    {"role": "user", "content": "Explain a Python decorator in one line."}
]

result = chat(messages, max_new_tokens=200)
print(result[0]["generated_text"][-1]["content"])
```

The messages format is identical to Ollama's

Same ChatML roles from Topic 32 and Topic 39. Different library, same shape.

`pipeline` is the beginner-friendly wrapper — it downloads the model, loads the tokenizer, formats the prompt, runs inference, and decodes the output, all in one call.

Layer 3 — Details that matter

Where models actually live

```
hf cache scan          # what's on disk and how big
hf cache delete        # interactive cleanup
```

Default location: `~/.cache/huggingface/hub`. This grows **fast** — a few experiments and you're 50 GB deep. Move it if your C: drive is small:

```
export HF_HOME=/path/to/big/drive/huggingface
```

A pattern that keeps recurring

The cache is **content-addressed with symlinks**, so two models sharing a file store it once. Same idea as Ollama's blob store (Topic 36) and Docker's layers (Topic 35).

`.safetensors` VS `.bin`

Format	Safety
<code>.bin</code> (pickle)	Can execute arbitrary code on load. Legacy.
<code>.safetensors</code>	Pure data, cannot execute anything.

This is a real attack vector

Always prefer `.safetensors`. `transformers` picks it automatically when present. Since anyone can publish to the Hub, malicious `.bin` files have been found before.

The RAM problem

Gemma 3 4B is ~8 GB on disk in fp16, and needs roughly that in RAM/VRAM to run.

```
from transformers import AutoModelForCausalLM
import torch

# Half precision - half the memory
model = AutoModelForCausalLM.from_pretrained(
    "google/gemma-3-4b-it",
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
```

Or 4-bit quantization (needs `bitsandbytes`, NVIDIA GPU):

```
from transformers import BitsAndBytesConfig

quant = BitsAndBytesConfig(load_in_4bit=True)
model = AutoModelForCausalLM.from_pretrained(
    "google/gemma-3-4b-it",
    quantization_config=quant,
    device_map="auto",
)
```

The trade-off in one sentence

Same quantization concept from Topic 34 — but **Ollama chose the defaults for you; Hugging Face makes you choose.**

Gemma 3 is multimodal

The 4B and larger Gemma 3 models accept **images as well as text**:

```
messages = [
  {
    "role": "user",
    "content": [
      {"type": "image", "url": "https://example.com/chart.png"},
      {"type": "text", "text": "What does this chart show?"},
    ],
  }
]
```

The `content` field becomes a **list of typed blocks** instead of a plain string.

Common errors, decoded

Error	Cause	Fix
<code>401 Unauthorized</code>	Not logged in	<code>hf auth login</code>
<code>403 Forbidden</code>	Logged in, license not accepted	Accept it on the model page
<code>GatedRepoError</code>	Same as above	Accept the license
<code>hf: command not found</code>	PATH not refreshed	Restart terminal; check <code>pip show huggingface_hub</code>
<code>huggingface-cli: deprecated</code>	Old command	Use <code>hf</code>
<code>OutOfMemoryError</code>	Model too big for RAM/VRAM	<code>bfloat16</code> , 4-bit, or a smaller model
<code>Repository Not Found</code>	Typo, or private repo	Check exact name; check you're logged in

✅ The distinction worth memorizing

401 = "who are you?" · 403 = "I know who you are, but you haven't signed the license."

Docker and tokens

```
app:
  build: .
  environment:
    - HF_TOKEN=${HF_TOKEN}      # read from your shell / .env
  volumes:
    - hf_cache:/root/.cache/huggingface  # don't re-download 8 GB

volumes:
  hf_cache:
```

🔥 Never bake a token into an image

It lands in a layer and is readable via `docker history` (Topic 35). And that volume line is the same lesson as `ollama_data` — without it, every container restart re-downloads the model.

Q: What is Hugging Face and what problem does it solve?

It's a hub for machine learning artifacts — models, datasets, and hosted demos — plus the libraries to use them, principally transformers. It solved a real fragmentation problem: before it, model weights were scattered across university FTP servers and Google Drive links with inconsistent loading code. Now there's one namespace, one API, and versioned repositories with a git-like model.

Q: What are weights, in plain terms?

The trained numerical parameters of a model — billions of floating-point values that encode everything it learned during training. Architecture is the code describing how data flows through the network; weights are the learned values that flow through it. Two models with identical architecture behave completely differently based on weights alone, which is why the weights file is the valuable artifact and why model size is quoted in parameters.

Q: What is a gated model, and how do you access one?

A model whose publisher requires accepting a license before download — Gemma, Llama, and others use this. You accept on the model's page while logged in, then authenticate locally with a token so the download carries your identity. It's free; the gating exists for acceptable-use agreement and export-control record-keeping. The failure mode is a 403, distinct from a 401, which means you're not authenticated at all.

Q: Difference between a base model and an instruction-tuned model?

A base model is trained only on next-token prediction — it continues text but doesn't understand instructions or conversational roles. An instruction-tuned model has been further fine-tuned on instruction-response pairs and chat transcripts, so it responds to system/user/assistant structure. The `-it` or `-instruct` suffix marks it. For any application you want the instruction-tuned version; base models are starting points for your own fine-tuning.

Q: Why prefer `.safetensors` over `.bin`?

`.bin` files use Python pickle, which can execute arbitrary code during deserialization — loading an untrusted one is remote code execution. `.safetensors` is a pure data format with no code path, so loading a malicious file can't do anything beyond producing wrong tensors. Since the Hub allows anyone to publish, this matters in practice.

Q: When would you use Hugging Face instead of Ollama?

Ollama when you want a chat model running locally with minimal setup and an HTTP API. Hugging Face when you need something Ollama doesn't carry — a domain-specific fine-tune, an embedding model, a vision or speech model — or when you need to fine-tune, control quantization and generation parameters precisely, or embed inference in a Python process rather than call out to a server. They're complementary: many Ollama models are GGUF conversions of Hugging Face repos.

MEMORY HOOK

Hugging Face is GitHub for models. `owner/model-name`, same as `owner/repo`.

The CLI is `hf`, not `huggingface-cli` → `hf auth login`

401 = who are you? · 403 = sign the license.

Architecture is the recipe; weights are the ingredients.

